



MANUAL: Model 980A Computer
FORTRAN Library and In-Line Assembly Language (961961-9744)

ECN NO: 384435 (B Level)

DATE: 15 March 1974

This change package reflects revisions to the manual as a result of miscellaneous changes in routines. The following changed pages should be inserted into the present manual in place of the existing pages:

<u>Manual</u> <u>Page</u>	<u>Change</u>
Cover/Effective Pages	Replace old cover and copyright/ECN page with new cover and new copyright/list of effective pages record.
2-7	*ARGADR to ARGADR in subroutine.
2-23	DLG10 to DLOG10 in paragraph 2-6.2.7.
4-2	Changed method of indicating indirect addressing.
4-3	Changed indirect addresses in subprogram example.
A-2	DINT to IDINT.
Response Sheet/ Business Reply	Replace old Comment Sheet with new Response Sheet.
Back Cover	Replace burnt orange back cover with green one.

MODEL 980A COMPUTER FORTRAN LIBRARY AND IN-LINE ASSEMBLY LANGUAGE

MANUAL NO. 961961-9744
ORIGINAL ISSUE 2 OCTOBER 1972
REVISED AND REISSUED 1 MARCH 1973

CHANGE 1

15 MARCH 1974

JUN 15 1975

NATIONAL BIOMEDICAL RESEARCH FOUNDATION
Georgetown University Medical Center
3900 Reservoir Road, N. W.
Washington, D. C. 20007



TEXAS INSTRUMENTS
INCORPORATED

© Texas Instruments Incorporated 1974

All Rights Reserved

The information and/or drawings set forth in this document and all rights in and to inventions disclosed herein and patents which might be granted thereon disclosing or employing the materials, methods, techniques or apparatus described herein are the exclusive property of Texas Instruments Incorporated.

No disclosure of the information or drawings shall be made to any other person or organization without the prior consent of Texas Instruments Incorporated.

INSERT LATEST CHANGED PAGES DESTROY SUPERSEDED PAGES

LIST OF EFFECTIVE PAGES

Note: The portion of the text affected by the changes is indicated by a vertical line in the outer margins of the page.

Original Issue October 1972
Revised and Reissued March 1973 (ECN 373977)
Change 1 March 1974 (ECN 384435)

Total number of pages in this publication is 78 consisting of the following:

Page No.	Change No.	Page No.	Change No.	Page No.	Change No.
Title	1	4-1	0		
A	1	4-2 - 4-3	1		
iii - v	0	4-4 - 4-6	0		
vi Blank	0	A-1	0		
1-1	0	A-2	1		
1-2 Blank	0	A-3	0		
2-1 - 2-6	0	A-4 Blank	0		
2-7	1	B-1	0		
2-8 - 2-22	0	B-2 Blank	0		
2-23	1	Response Sheet . .	1		
2-24 - 2-41	0	Business Reply . .	1		
2-42 Blank	0	Cover Blank	0		
3-1 - 3-10	0	Cover Imprint . . .	0		

- d. XIT1\$ – When a subroutine which initially calls ARG\$ at SET1\$ is finished, it should exit to XIT1\$. This causes the value of MODE\$ to be restored and returns control to the main program.
- e. XIT2\$ – Serves the same purpose as XIT1\$ for subroutines which initially call ARG\$ at SET2\$.

To illustrate the use of ARG\$, consider the construction of a subroutine which forms the sum of a number of floating-point numbers. Let the entry point of the subroutine be SUM. The calling sequence is:

```

BRL    *$+1
DATA   SUM
DATA   N          ARGUMENT COUNT
DATA   Address of First Argument
      .
      .
      .
DATA   Address of Last Argument

```

The subroutine would appear

SUM	RMO	L,A	LINK REGISTER TO A
	LDE	=0	SET MODE\$ TO SINGLE-PRECISION
	BRL	*SET1	GO TO ARG\$ AT SET1\$
	STA	ARGADR	ADDRESS OF FIRST ARGUMENT
	RCO	M,X	NEGATIVE OF ARG COUNT TO X
	FLD	*ARGADR	LOAD FIRST ARGUMENT
	BRU	BIX	
LOOP	BRL	*ARG	GET NEXT ARGUMENT ADDRESS FROM
	STA	ARGADR	ARG\$ - ENTRY POINT ARG\$
	FAD	*ARGADR	ADD NEXT ARGUMENT
BIX	BIX	LOOP	BUMP ARGUMENT COUNT
	BRU	*XIT1	ALL ARGUMENTS USED - EXIT TO
ARGADR	BSS	1	ARG\$ AT XIT1\$
XIT1	DATA	XIT1\$	
ARG	DATA	ARG\$	
SET1	DATA	SET1\$	
	REF	ARG\$, SET1\$, XIT1\$	

If the calling sequence does not contain an argument count, different entry points to ARG\$ are used. Suppose the larger of two double-precision quantities must be determined, and it must be left in the floating accumulator. The calling sequence is:

BRL	*\$+1	
DATA	MAX	
DATA	ARG1	ADDRESS OF FIRST ARGUMENT
DATA	ARG2	ADDRESS OF SECOND ARGUMENT

The subroutine MAX would appear as follows:

MAX	RMO	L,A	LINK TO A
	LDE	=2	SET MODE\$ FOR DOUBLE-PRECISION
	BRL	*SET	
	STA	ARGADR	ADDRESS OF FIRST ARGUMENT
	FLD	*ARGADR	FIRST ARGUMENT TO FLOATING ACC.
	BRL	*ARG	FETCH AND STORE ADDRESS
	STA	ARGADR	OF SECOND ARGUMENT
	FCP	*ARGADR	COMPARE ARGUMENTS
	SLT		
	BRU	*XIT	FIRST ARGUMENT GREATER - EXIT
	FLD	*ARGADR	SECOND ARGUMENT GREATER - PLACE IN FLOATING ACC.
	BRU	*XIT	
SET	DATA	SET2\$	
ARGADR	BSS	1	
XIT	DATA	XIT2\$	
ARG	DATA	ARG\$	
	REF	SET2\$, XIT2\$, ARG\$	

2-3.3 BASIC EXTERNAL FUNCTIONS. Appendix Table A-1 summarizes the basic external functions available in the FORTRAN library. These subroutines are available to both the FORTRAN and assembly language programmer. Section 3-2 specifically covers their use in assembly language programs. The subprograms included in this group are characterized by the fact that they implement certain commonly recognized mathematical functions.

2-3.4 INTRINSIC FUNCTIONS. Appendix Table A-2 summarizes the intrinsic functions available in the FORTRAN library. These subroutines are also available to both the FORTRAN and assembly language programmer. Section 3-2 specifically covers their use in assembly language programs. The subprograms included in this group are characterized by the fact that they implement certain useful logical procedures.

$$f(z) = \sum_{n=0}^5 a_n z^n$$

and

$a_0 = 1.0$
 $a_1 = 0.69314\ 7181$
 $a_2 = 0.24022\ 6486$
 $a_3 = 0.05550\ 41054$
 $a_4 = 0.00962\ 173987$
 $a_5 = 0.00133\ 37729375$

therefore $e^x = 2^{\frac{x}{\ln 2}} = 2^{\frac{d}{f(z)}}$

2-6.2.6 ARCTANGENT OF TWO ARGUMENTS.

Name DATN2\$

Entry DATAN2

The arctangent of the ratio of the two arguments is computed using DATN2\$. The result is modified according to the signs of the arguments to produce an angle in the range $(-\pi, \pi)$ as shown below.

Arg 1	Arg 2	Result
+	+	No modification
+	-	Add π
-	+	No modification
-	-	Sub π

2-6.2.7 BASE TEN LOGARITHM.

Name DLN10\$

Entry DLOG10

The natural logarithm is computed using DLOG\$ and the result is multiplied by $\log_{10} e$.

2-6.3 COMPLEX FUNCTIONAL ROUTINES.

The calling sequence to all complex functional routines is

```

BRL    *$+1
DATA   Subroutine Entry
DATA   1
DATA   Argument Address

```

All leave the result in CACC\$, CACC2\$.

2-6.3.1 COMPLEX SQUARE ROOT.

Name CSQRT\$

Entry CSQRT

Let the argument be represented in rectangular form $x+iy$. Then

$$(x + iy)^{1/2} = (x^2 + y^2)^{1/2} e^{i\theta/2} = R e^{i\theta/2} = R(\cos \theta/2 + i \sin \theta/2)$$

where

$$\theta/2 = 1/2 \arctan (y/x)$$

The required function is computed by evaluating $R(\cos \theta/2 + i \sin \theta/2)$ and the result is left in the complex accumulator. CSQRT\$ requires ARGSU\$, ATAN\$, ALOG\$, SINCOS, and EXP\$, in addition to CARIT\$.

2-6.3.2 COMPLEX SINE/COSINE.

Name CSICO\$

Entry CSIN, CCOS

The sine and cosine of complex quantities are determined from the identities.

$$\begin{aligned} \sin(x+iy) &= \sin x \cos iy + \cos x \sin iy \\ &= \sin x \cosh y + i \cos x \sinh y \end{aligned}$$

$$\begin{aligned} \cos(x+iy) &= \cos x \cos iy - \sin x \sin iy \\ &= \cos x \cosh y - i \sin x \sinh y \end{aligned}$$

$$\sinh y = (e^y - e^{-y})/2$$

$$\cosh y = (e^y + e^{-y})/2$$

2-6.3.3 COMPLEX LOGARITHM.

Name CLOG

Entry CLOG

The complex logarithm is computed from the relation

$$\ln(x+iy) = 1/2 \ln(x^2 + y^2) + i \arctan y/x$$

CLOG requires ARGSU\$, ATAN, and ALOG.

SECTION IV

IN-LINE ASSEMBLY LANGUAGE STATEMENTS

4-1 SCOPE.

In addition to the standard FORTRAN statements defined in the preceding sections, the 980A FORTRAN has been extended to accept a rather large subset of the 980A assembly language mnemonics. This subset is referred to as In-Line Assembly Language (IAL). When the mnemonics are used in a FORTRAN program, the first character of the line (or column 1 of the card) must be the letter A. A four-digit, unsigned number, in characters 2 through 5, is allowed as a statement number. Characters in position 6 are ignored. If a statement is followed by a semicolon, all characters appearing after the semicolon are treated as comments. Comments may be inserted also by using the letter C as the first character of the line.

In IAL statements, the symbols > or \ preceding a digital string identify that string as a hexadecimal number.

4-2 REGISTER-TO-MEMORY INSTRUCTIONS.

The following instructions of the register-to-memory type are allowed. They may be preceded by @ which indicates extended format. The mnemonic codes have the meaning described in the Assembly Language Machine Instructions manual.

ADD	DIV	LDE
AND	DLD	LDM
BIX	DMT	LDX
BRL	DSB	MPY
BRU	DST	STA
CPA	IMO	STE
CPL	IOR	STX
DAD	LDA	SUB

These instructions have two operands. The use of these operands is exactly as with ordinary assembly language programming. The first operand of a register-to-memory instruction may be a scalar or array variable name, plus or minus a decimal integer constant less than or equal to 32,767 or a null field. The first operand may also be a decimal or hexadecimal integer constant. If the instruction is not extended format, this may be a decimal constant in the range -128 to +127, inclusive, or a hexadecimal constant from -\80 to +\7F, inclusive. If the extended format is used, the number may be in the range -32,768 to +32,767 decimal or 8000 to 7FFF hexadecimal. Labels appearing on FORTRAN statements may also serve as the first operand. This is indicated by a preceding ampersand (&) and may have a decimal constant value added or subtracted. Finally, the present value of the location counter designated by \$ may appear as the first operand with an optional decimal constant added or subtracted. The second operand is an addressing mode identifier (IXB field) or a null field. If the IXB field is null, the comma after the first operand field may be omitted.

The allowable mode identifiers are as follows:

IDENTIFIER	VALUE	NAME
BR	1	Base register relative
X	2	Indexed, program counter relative
XBR	3	Indexed, base register relative
I	4	Indirect, program counter relative
IBR	5	Indirect, base register relative
IX	6	Indirect, post indexing, program counter relative
M	7	Immediate addressing

The first operand may optionally be followed by ,I which indicates indirect addressing.

Examples:

```
A      LDA      0,M
A      LDX      -17,M
A      LDE      >8D,M
A      @LDA     ->8000,M
A      @LDM     >FF0,M
A 603  LDA      ITEM
A      MPY      3,M
A      LDM      IPART+17,X
A      LDA      REAL,BR
A      LDE      REAL+1,BR
A      BIX      &605
A      BRU      &45-2,X
A      BRU      $-6
A      LDA      $+25,IX
```

When NAME is referenced within a FORTRAN function or subroutine subprogram, and NAME is one of the dummy parameters appearing in the subroutine or function definition statement, the operand acquired is the *address* of the actual parameter as it appears in the subprogram reference, and not the *value* of the parameter itself.

Example:

```

      INTEGER FUNCTION RAND (J,L)
A      LDA      J
A      STA      &5
A      LDA      L
A      STA      &10
A      LDA      &10,I
A      AND      &5,I
A      STA      RAND
      RETURN
A      5      DATA      0
A      10     DATA      0
      END
```

NOTE

The only valid method to return from a subprogram is by use of the RETURN statement.

In the case of function subprograms, the compiler generates code to load the proper registers with the function value prior to generating the calling sequence for the return. Hence, the assembly language programmer must *store* all function values before writing the RETURN statement. They may not be left in registers.

4.3 REGISTER-TO-REGISTER INSTRUCTIONS.

The following instructions of the register-to-register type are allowed:

RSU	RIN	RCL
RAD	RCA	RAN
RCO	ROR	RDE
RIV	RMO	REX
REO		

As operands for the instructions, the following symbols may be used to identify source and destination registers.

A - Accumulator register
E - Extension register
X - Index register
M - Mask register
S - Storage register
L - Link register
B - Base register
P - Program register

4-4 INDICATOR SKIP INSTRUCTIONS.

The following are allowed:

SEQ	SGE	SNV
SNE	SLE	SOC
SGT	SOV	SNC
SLT		

No operands are allowed for these instructions.

4-5 REGISTER SKIP INSTRUCTIONS.

All of the register skip types are allowed.

SZE	SMI	SOO
SNZ	SEV	SNO
SPL	SOD	

The allowable operand is one of the register symbols listed under 4-3.

4-6 SHIFT INSTRUCTIONS.

The following are allowed:

ARA	LLD	CRX
ARD	RTO	CRM
LRA	RTZ	CRS
LRD	LTO	CRL
ALA	LTZ	CRB
ALD	CRA	CLD
LLA	CRE	CRD

The allowable operand is a number less than 32, indicating the shift count.

APPENDIX A

TABLE A-1. FORTRAN BASIC EXTERNAL FUNCTIONS

Function	Definition	No. Arguments	Name	Arg. Type	Function Type
Exponential	e^a	1	EXP	Real	Real
			DEXP	Double	Double
			CEXP	Complex	Complex
Natural Log	$\log_e(a)$	1	ALOG	Real	Real
			DLOG	Double	Double
			CLOG	Complex	Complex
Common Log	$\log_{10}(a)$	1	ALOG10	Real	Real
			DLOG10	Double	Double
Sine	$\sin(a)$	1	SIN	Real	Real
			DSIN	Double	Double
			CSIN	Complex	Complex
Cosine	$\cos(a)$	1	COS	Real	Real
			DCOS	Double	Double
			CCOS	Complex	Complex
Hyperbolic Tan	$\tanh(a)$	1	TANH	Real	Real
Square Root	$(a)^{1/2}$	1	SQRT	Real	Real
			DSQRT	Double	Double
			CSQRT	Complex	Complex
Arctangent	$\arctan(a)$	1	ATAN	Real	Real
		1	DATAN	Double	Double
	$\arctan(a_1/a_2)$	2	ATAN2	Real	Real
		2	DATAN2	Double	Double
Remaindering*	$a_1 \text{ (mod } a_2)$	2	DMOD	Double	Double
Modulus	$ c $	1	CABS	Complex	Real

*The function DMOD (a_1, a_2) is defined as $a_1 - [a_1/a_2] a_2$ when $[a_1/a_2]$ is the integer whose magnitude does not exceed $|a_1/a_2|$.

TABLE A-2. FORTRAN INTRINSIC FUNCTIONS

Function	Definition	No. Args.	Name	Arg. Type	Function Type
Absolute Value	$ a $	1	ABS	Real	Real
			IABS	Integer	Integer
			DABS	Double	Double
Truncation	Sign of a times largest integer $\leq a $	1	AINT	Real	Real
			INT	Real	Integer
			IDINT	Double	Integer
Remaindering*	$a_1 \text{ (mod } a_2 \text{)}$	2	AMOD MOD	Real Integer	Real Integer
Choose Largest Value	$\text{MAX}(a_1, a_2, \dots, a_n)$	≥ 2	AMAX0	Integer	Real
			AMAX1	Real	Real
			MAX0	Integer	Integer
			MAX1	Real	Integer
			DMAX1	Double	Double
Choose Smallest Value	$\text{MIN}(a_1, a_2, \dots, a_n)$	≥ 2	AMIN0	Integer	Real
			AMIN1	Real	Real
			MIN0	Integer	Integer
			MIN1	Real	Integer
			DMIN1	Double	Double
Float	Convert Integer to Real	1	FLOAT	Integer	Real
Fix	Convert Real to Integer	1	IFIX	Real	Integer
Transfer of Sign	Sign a_2 times $ a_1 $	2	SIGN	Real	Real
			ISIGN	Integer	Integer
			DSIGN	Double	Double
Positive Difference	$a_1 - \text{MIN}(a_1, a_2)$	2	DIM	Real	Real
			IDIM	Integer	Integer
Double to Real		1	SNGL	Double	Real
Real Part of Complex		1	REAL	Complex	Real
Imaginary Part of Complex		1	AIMAG	Complex	Real
Real to Double		1	DBLE	Real	Double
Two Reals to Complex	$a_1 + a_2 \sqrt{-1}$	2	CMPLX	Real	Complex
Complex Conjugate	$a - ib$	1	CONJG	Complex	Complex

*See footnote at bottom of Table A-1.

USER'S RESPONSE SHEET

Model 980A Computer FORTRAN Library and
Manual Title: In-Line Assembly Language 961961-9744

Date of Manual: 15 March 1974 (Change 1) Date of This Letter: _____

User: _____ Office/Dept. No.: _____

Company: _____

Street Address: _____

City/State/Zip: _____

Please list any discrepancy found in this manual by page, paragraph, figure, or table number in the following space. If there are any other suggestions that you wish to make, feel free to include them. Thank you.

Location
in Manual

Comment/Suggestion

NO POSTAGE NECESSARY IF MAILED IN U. S. A.

FOLD ON TWO LINES (LOCATED ON REVERSE SIDE), STAPLE AND MAIL

First Class
PERMIT NO. 3135
Austin, Texas

BUSINESS REPLY MAIL

No Postage Necessary if Mailed in the United States

Postage Will Be Paid by

TEXAS INSTRUMENTS INCORPORATED

DIGITAL SYSTEMS DIVISION

P.O. BOX 2909 · AUSTIN, TEXAS 78767

Attn: TECHNICAL PUBLICATIONS, MS 2146
